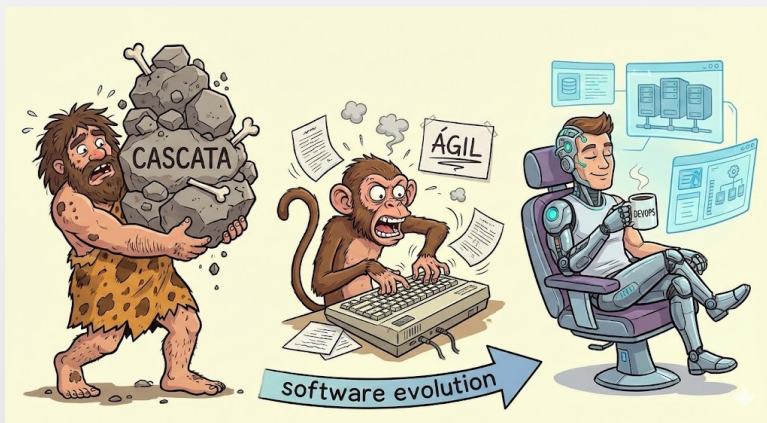


# DevOps



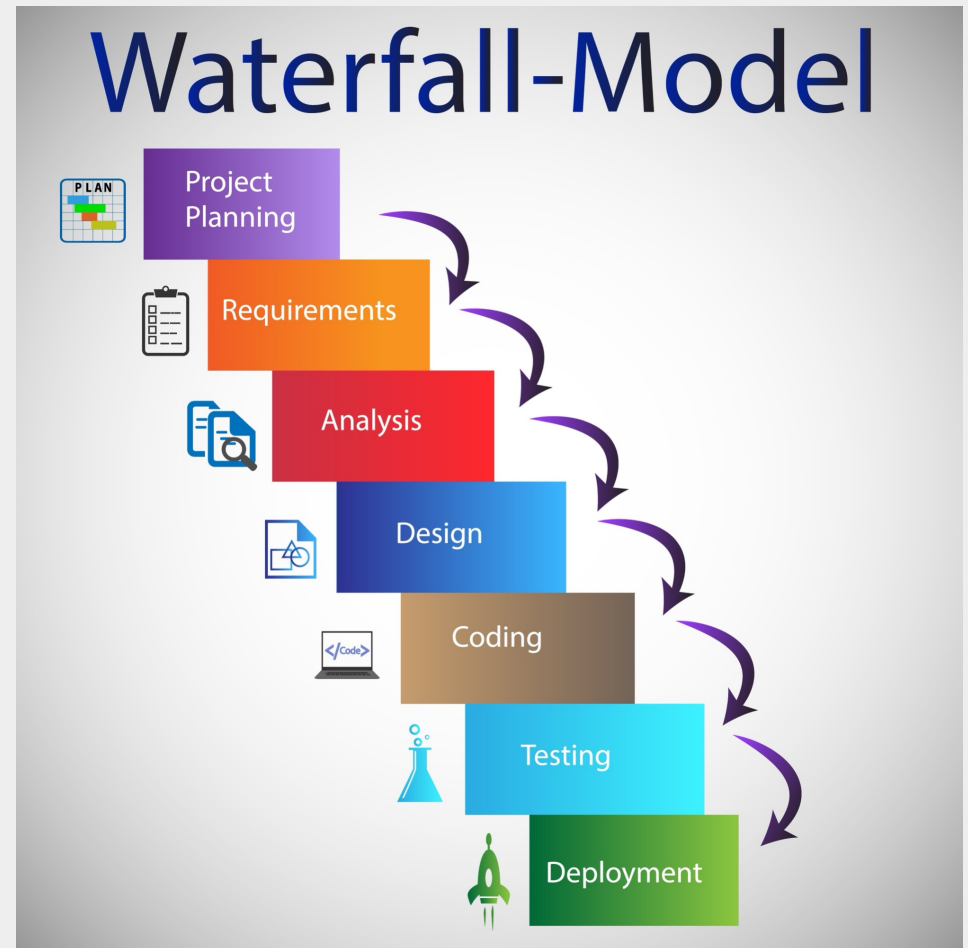
**Aula: Introdução ao  
DevOps: A Evolução da  
Cultura e Engenharia de  
Software**

# A Era do Desenvolvimento em Cascata (Waterfall)

O modelo tradicional de engenharia, herdado da construção civil.

Fases rígidas: Requisitos -> Design -> Implementação -> Verificação -> Manutenção.

Discussão: Por que isso falhava no software?



# A Era do Desenvolvimento em Cascata (Waterfall)

O modelo em Cascata, herdado das disciplinas de engenharia civil e manufatura, pressupunha que o software poderia ser construído em fases estanques e sequenciais: Levantamento de Requisitos, Design Arquitetural, Implementação, Verificação e Manutenção.

A transição entre essas fases era marcada por assinaturas de documentos extensos. O grande problema desse modelo na engenharia de software é a sua premissa de que os requisitos de negócios são imutáveis durante o ciclo de vida do projeto, o que na prática corporativa é uma falácia.

# O Surgimento do Ágil

## Manifesto Ágil (2001) - Resolvendo o lado do "Dev"

Foco em indivíduos e interações, software em funcionamento, colaboração com o cliente e resposta a mudanças. O Ágil acelerou drasticamente a forma como o código era escrito e como as features eram pensadas.



# O Surgimento do Ágil

## Manifesto Ágil (2001) - Resolvendo o lado do "Dev"

Em 2001, o Manifesto Ágil revolucionou a etapa de construção do software ao priorizar "software em funcionamento" e "resposta a mudanças" sobre o seguimento de planos rígidos. Metodologias como Scrum e Kanban introduziram ciclos iterativos e incrementais (sprints). A equipe de Desenvolvimento (Dev) passou a ter autonomia para alterar o escopo, refatorar o código e gerar novos artefatos funcionais a cada duas semanas, acelerando drasticamente o ritmo de inovação.

O Ágil resolveu o problema da criação do código, mas introduziu um novo vetor de pressão na ponta final da esteira: como colocar esse volume de mudanças no ar com segurança?

# O Gargalo Muda de Lugar

O desenvolvimento (Dev) adota Scrum/Kanban e produz versões a cada duas semanas.

Discussão: O que acontece com a Infraestrutura (Ops)? A operação continuava trabalhando no modelo tradicional, focada na estabilidade e na aversão a mudanças.



# O Gargalo Muda de Lugar

Enquanto o time de Desenvolvimento adotava o Ágil para maximizar a taxa de mudança, o time de Operações de TI (Ops) permanecia operando sob premissas tradicionais (como o ITIL clássico), focado primordialmente na estabilidade do ambiente de produção. O resultado imediato foi um acúmulo de artefatos "prontos" gerados pelo Dev, mas que aguardavam longas janelas de aprovação, comitês de mudança (CABs) e processos manuais para serem implantados em servidores.

O gargalo do fluxo de valor deixou de ser a escrita do código e passou a ser o processo de implantação e liberação (Release & Deploy).

# A Anatomia e as Métricas dos Silos Corporativos

**Dev:** Medido pela quantidade de novas funcionalidades entregues (Inovação/Mudança).

**Ops:** Medido pelo Uptime e estabilidade do sistema (Previsibilidade/Risco Zero).

Como toda mudança introduz risco, a Operação cria burocracias para impedir o Desenvolvimento de alterar o ambiente.





# O "Muro da Confusão" (Wall of Confusion)

O código é desenvolvido na máquina local. O desenvolvedor "joga" o pacote compilado por cima do muro para a operação implantar em produção.

A total ausência de comunicação empobrece a qualidade e gera o famoso jogo de empurra (Blame Game) durante incidentes.



# O "Muro da Confusão" (Wall of Confusion)

O desalinhamento de metas gera o "Muro da Confusão". O time de desenvolvimento escreve o código isoladamente, empacota o binário e "joga por cima do muro" para que a operação, que não participou da concepção da arquitetura, tente executá-lo. Quando ocorre uma degradação no serviço durante a implantação, a falta de contexto gera a cultura do apontamento de dedos (Blame Game), onde Dev culpa a infraestrutura e Ops culpa a qualidade do código.

O isolamento reduz a empatia profissional. O desenvolvedor perde o senso de responsabilidade sobre a execução do software, focando apenas na entrega do artefato.



# A Síndrome do "Na Minha Máquina Funciona"

A separação física e lógica das equipes resulta em divergência estrutural (Configuration Drift) entre os ambientes. O desenvolvedor escreve código baseando-se nas variáveis, versões de bibliotecas e no sistema operacional de seu notebook. A operação tenta rodar esse mesmo artefato em servidores com topologias de rede estritas, proxies, bancos de dados clusterizados e restrições de segurança distintas.

Sem padronização, a homologação de um sistema perde seu valor estatístico, pois o ambiente de testes não reflete fielmente o ambiente produtivo. O deploy se torna um evento traumático de madrugada ou de fim de semana, com alta taxa de falha.



# O Impacto Estratégico e Financeiro

O atrito técnico se traduz diretamente em **perda de competitividade corporativa**.

**Implantações traumáticas**, que exigem madrugadas ou finais de semana de trabalho manual, possuem altíssimas taxas de falha (Change Failure Rate).

O **tempo de recuperação diante de incidentes (MTTR)** é prolongado devido à falta de familiaridade de Ops com o código e de Dev com o servidor



# O Ponto de Virada: Velocity Conference 2009



O movimento **DevOps** ganhou tração formal a partir da apresentação paradigmática "10+ Deploys Per Day: Dev and Ops Cooperation at Flickr", conduzida por **John Allspaw e Paul Hammond**.

Eles demonstraram empiricamente que a **união** entre as **ferramentas adequadas**, **métricas compartilhadas** e **comunicação transparente** poderia **quebrar o paradigma de que velocidade e segurança são mutuamente excludentes**. Ao colaborar estreitamente, o Flickr conseguiu realizar **múltiplas implantações diárias** em uma época em que o **mercado levava meses**.

A automação deixou de ser vista como um **risco** e **passou a ser reconhecida como o principal vetor para implantações consistentes e auditáveis**.

# Desmistificando o Conceito de DevOps

É imperativo compreender o que DevOps **NÃO** é para evitar as armadilhas comuns do mercado.

DevOps **NÃO** é um título de cargo (embora o mercado use o termo "Engenheiro DevOps" por conveniência),

**NÃO** é um departamento isolado (criar uma "equipe de DevOps" frequentemente gera apenas um terceiro silo no fluxo) e, acima de tudo,

**NÃO** é um produto de prateleira que se compra. **Nenhuma ferramenta, isoladamente, garante a adoção dessa cultura.**

Ferramentas facilitam práticas, mas sem a mudança da estrutura colaborativa da empresa, elas se tornam apenas implementações caras de processos engessados.



# Definição Formal e o Fluxo de Valor

**DevOps** é a integração multidisciplinar de pessoas, processos e tecnologias cujo objetivo primário é fornecer **valor contínuo e mensurável ao usuário final**.

É uma filosofia de engenharia que busca **encurtar significativamente o Ciclo de Vida de Desenvolvimento de Sistemas (SDLC)**, garantindo **qualidade, segurança intrínseca** e um **ciclo de feedback ininterrupto**.

O foco é a otimização holística do **Fluxo de Valor (Value Stream)**.

O Fluxo de Valor mapeia cada etapa desde a ideação do requisito de negócio até o instante em que a funcionalidade roda em produção gerando receita.



# A Herança do Lean Manufacturing

Os princípios do DevOps derivam profundamente do Sistema Toyota de Produção e da filosofia Lean. A ênfase é na identificação e eliminação sistemática de desperdícios (Kaizen) — sejam eles tempos de espera em aprovações, over-engineering de código ou movimentações desnecessárias de pacotes.

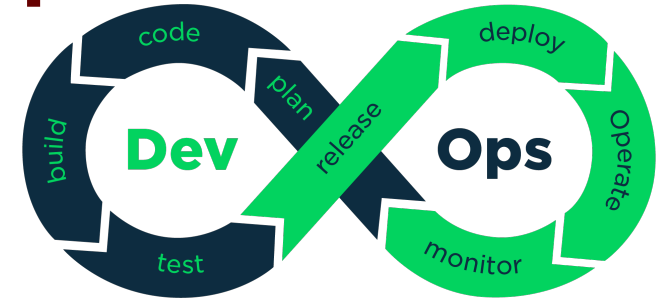
Incorpora-se o conceito de "Andon": a capacidade e o dever de qualquer membro da equipe ou ferramenta automatizada de parar a linha de produção (quebrar o pipeline) assim que um defeito de qualidade é detectado.

A qualidade deve ser construída no processo (built-in quality), e não inspecionada apenas na fase final de testes manuais.





# O Ciclo Infinito do DevOps e Feedback Contínuo



O símbolo do infinito representa a natureza perpétua da entrega de software, eliminando a ideia de que um software fica "pronto". O ciclo integra o planejamento ágil (Plan), codificação (Code), automação de compilação (Build), testes contínuos (Test), gestão de releases (Release), implantação padronizada (Deploy), gerenciamento de infraestrutura (Operate) e observabilidade profunda (Monitor).

A fase de Monitoramento não é o fim, mas a origem dos dados analíticos que baseiam e priorizam o planejamento do próximo ciclo de iteração.

# O Acrônimo CAMS (A Essência da Transformação)

Cunhado por John Willis e Damon Edwards, o acrônimo CAMS sintetiza os pilares da adoção.

**Cultura** (**C**ulture): Resolução de conflitos e empatia estrutural.

**Automação** (**A**utomation): Remoção sistemática do trabalho humano repetitivo.

**Medição** (**M**easurement): Tomada de decisão baseada em telemetria, não em intuição.

**Compartilhamento** (**S**haring): Transparência no código, nos sucessos e nas falhas para elevar o nível técnico de toda a corporação.

Qual destes quatro pilares é historicamente o mais difícil de ser implementado em empresas tradicionais? (Dica: não é a tecnologia).



# O Princípio "You Build It, You Run It"

Popularizado por Werner Vogels (CTO da Amazon), este princípio determina a responsabilidade integral pelo serviço.

Desenvolvedores não apenas escrevem a lógica de negócio, mas suportam a operação de seu código em produção.

Essa mudança de paradigma alinha perfeitamente os incentivos: se um engenheiro souber que será acordado de madrugada caso seu código cause um vazamento de memória, a atenção à qualidade arquitetural, ao tratamento de exceções e aos testes será maximizada durante a construção.

A propriedade total do serviço **destrói** o modelo mental do "Muro da Confusão".



# Cultura Blameless (A Anatomia do Erro em Sistemas Complexos)

Em sistemas de TI distribuídos e hiperconectados, falhas catastróficas raramente são o resultado de um único evento; elas ocorrem quando múltiplas pequenas anomalias se alinham.

A cultura "Blameless" (Livre de Culpa) parte do princípio de que os profissionais são bem-intencionados e agem com base nas informações que possuem no momento. Quando uma falha ocorre, a investigação ignora "quem" errou e foca rigorosamente em descobrir "onde o sistema falhou" ao permitir que o humano causasse o dano.

Punir indivíduos esconde incidentes. Ajustar os bloqueios sistêmicos (guardrails) previne a reincidência.



# Segurança Psicológica e Taxa de Inovação

Equipes submetidas ao **medo constante de causar incidentes operacionais** desenvolvem aversão crônica ao risco, o que se traduz em pacotes de implantação colossais realizados com **meses de intervalo**.

Paradoxalmente, **pacotes maiores** contêm mais linhas de código desconhecidas, elevando **drasticamente a chance de falha**.

Equipes com alta segurança psicológica implementam rotinas de **implantação frequentes e granulares** (dezenas por dia), **reduzindo o escopo** (blast radius) de cada mudança e **facilitando a recuperação imediata**.

A agilidade corporativa depende diretamente do nível de confiança e segurança estrutural que a engenharia possui para falhar pequeno e rápido.



# Paradoxo do Toil (Trabalho Massante e Operacional)

Na engenharia de confiabilidade (SRE/DevOps), "Toil" é definido como o trabalho manual, tático, desprovido de valor duradouro de engenharia, repetitivo e, criticamente, que escala linearmente conforme a infraestrutura cresce.

Exemplos incluem: reiniciar serviços travados, limpar discos manualmente ou criar usuários em dezoito bancos de dados diferentes a cada contratação. O Toil drena o tempo da equipe técnica, impedindo que atuem na construção de arquiteturas resilientes e automatizadas.

Um dos objetivos primários da automação em DevOps é limitar o Toil a, no máximo, 50% do tempo do engenheiro, liberando o resto para inovação sistêmica.



# O Paradigma "Tudo como Código" (Everything as Code - EaC)

O avanço tecnológico permitiu que as abstrações de hardware se tornassem APIs puras. Dessa forma, as práticas fundamentais de desenvolvimento de software (versionamento, revisão por pares, testes automatizados e integração contínua) são aplicadas a toda a infraestrutura de TI.

Redes virtuais, regras de firewall, topologias de cluster e **pipelines** de **CI/CD** deixam de ser configurações manuais interativas em consoles visuais e passam a ser arquivos declarativos textuais armazenados em repositórios centrais.

A infraestrutura imutável garante que servidores não sofrem mutações lentas e indocumentadas ao longo dos meses.



# A Sopa de Letrinhas e o Pipeline de Engenharia



Para tangibilizar a teoria, o DevOps se apoia em fluxos técnicos automatizados:

**CI (Integração Contínua):** Mesclagem diária de código, validada por builds automatizados e milhares de testes unitários.

**CD (Entrega Contínua):** O artefato de software passa por toda a esteira de qualidade e repousa homologado, pronto para implantação na produção ao acionamento de um gatilho.

**IaC (Infraestrutura como Código):** Provisionamento instantâneo, seguro e auditável do ambiente de execução necessário para o artefato.

Estes elementos operam em conjunto: o código passa pelo **CI**, valida-se o negócio, e através do **CD**, é implantado na infraestrutura orquestrada via **IaC**.



# Observabilidade Profunda vs. Monitoramento Passivo

**Monitoramento tradicional** é binário e passivo: ele responde à pergunta "Meu servidor ou serviço está de pé agora?".

A **Observabilidade** é o salto evolutivo para ambientes distribuídos e dinâmicos. Ela unifica e correlaciona as três bases da telemetria: **Logs Estruturados**, **Métricas de Sistema/Negócio** e **Traces Distribuídos**, permitindo que os engenheiros respondam a perguntas abertas e complexas: "Por que os usuários usando iOS na região Sul estão experimentando latência de 2 segundos na rota de pagamento desde as 14h?".

A observabilidade não depende de instrumentação visual pontual, mas de código injetado que conta a história completa de cada requisição no sistema.



# Medindo Alta Performance com as Métricas DORA

Após quase uma década de pesquisa acadêmica e de mercado pelo grupo DevOps Research and Assessment (DORA), consolidaram-se as **4 métricas definitivas** que distinguem organizações de elite das estagnadas:

**Deployment Frequency:** Com que frequência o código entra em produção?

**Lead Time for Changes:** Qual o tempo total desde o 'commit' de uma linha de código até ela estar rodando em produção?

**Mean Time to Recovery (MTTR):** Quando há degradação crítica, quanto tempo leva para restabelecer o serviço?

**Change Failure Rate:** Qual a porcentagem de deploys que exigem reversão (rollback) ou hotfixes devido a anomalias introduzidas?

As duas primeiras medem Velocidade. As duas últimas medem Estabilidade. DevOps prova que o foco nas 4 gera melhoria orgânica em todos os eixos.



# Estudo de Caso: Fluxo Transacional Crítico

Considere um ecossistema de altíssima criticidade e exigência transacional, como as operações de processamento de meios de pagamento e cartões de crédito.

Nesse cenário, o serviço é responsável por receber a requisição de autorização da compra e realizar junto a múltiplas bandeiras e arranjos financeiros.

O SLA (Service Level Agreement) de resposta geralmente beira os milissegundos.

Considerando esse cenário, digamos em que um sábado o serviço responsável pela autorização ficou mais lento que o habitual, quais os riscos envolvidos?



# O Custo da Indisponibilidade no Modelo Tradicional

Se um bug é introduzido no serviço de autorização sob o paradigma tradicional: o código falho sobe sem testes estritos e começa a degradar.

Os clientes nas lojas físicas tentam passar os cartões e recebem "Time Out".

A fila de chamados de suporte incha. A equipe de operações de TI (N1) olha para dashboards de uso de CPU que mostram que tudo "está verde", pois o erro é lógico, não de processamento.

Horas se passam até que o suporte consiga escalar para a engenharia central analisar de forma manual os arquivos de texto isolados dentro dos servidores.

O Mean Time to Recovery (MTTR) neste cenário é catastrófico. O impacto à reputação do serviço é permanente.



# A Abordagem DevOps e a Observabilidade Ativa

Em uma organização com **cultura DevOps madura**, o mesmo **bug** comportaria-se de maneira diferente.

As esteiras de CI e CD implementam a entrega, e as ferramentas robustas de observabilidade tomam o controle.

Plataformas capturam, em tempo real, que a taxa de sucesso das autorizações do serviço de clearing caiu de 99,99% para 96% logo após a atualização.

O sistema não aguarda reclamação humana. Ele detecta a anomalia através de desvios nas métricas de negócio.

A telemetria proativa reverte a lógica: o sistema avisa o engenheiro sobre o erro antes mesmo que o volume crítico de usuários sofra os impactos.



# Resposta Rápida e Redução Drástica do MTTR

A plataforma de observabilidade aciona gatilhos inteligentes através de roteadores de alertas corporativos (como o OpsGenie).

O desenvolvedor que é o dono daquele serviço, que está de plantão (on-call), recebe imediatamente a notificação em seu dispositivo móvel, já com o link para o "Trace" da requisição exata que falhou e o histórico do log do contêiner associado.

Sem necessidade de investigar dezenas de painéis, a causa raiz temporal é associada instantaneamente ao último deploy feito minutos atrás.

O engenheiro aciona o **Rollback** automatizado no pipeline. **O serviço é restabelecido em minutos.**



# Aprendizado Institucional e Gestão do Conhecimento

O incidente foi estancado rapidamente (Baixo MTTR). O passo crítico do DevOps ocorre após a crise.

A equipe realiza uma cerimônia estruturada de "Post-Mortem" blameless.

O evento é minuciosamente documentado em sistemas de base de conhecimento compartilhada e rastreamento de tarefas (como Jira e Confluence).

A discussão foca em: "Qual regra de validação no nosso pipeline de CI falhou em detectar esse cenário antes de chegar à produção?".

Cria-se um novo teste unitário que simula a exata falha que ocorreu. Isso garante matematicamente que aquela categoria de erro jamais atinja a produção novamente, fechando o ciclo de melhoria contínua.



# Reflexão: A Interdependência das Ferramentas

Ao olhar para um ecossistema moderno que orchestra Jira (gestão de demandas), Confluence (documentação ágil), Github/Gitlab (versionamento corporativo), ferramentas de CI/CD robustas, IaC declarativa, e plataformas completas de monitoramento (Datadog integrado ao OpsGenie), notamos que **nenhuma delas isoladamente resolve o problema corporativo**. Elas operam como conectores que **amplificam a colaboração e viabilizam o modelo de operação segura**.

A ferramenta reflete o processo corporativo subjacente. Processos falhos com ferramentas modernas geram apenas gargalos mais caros.





# DevOps



**Controle de Versão e Git:  
A Máquina do Tempo da  
Engenharia de Software**

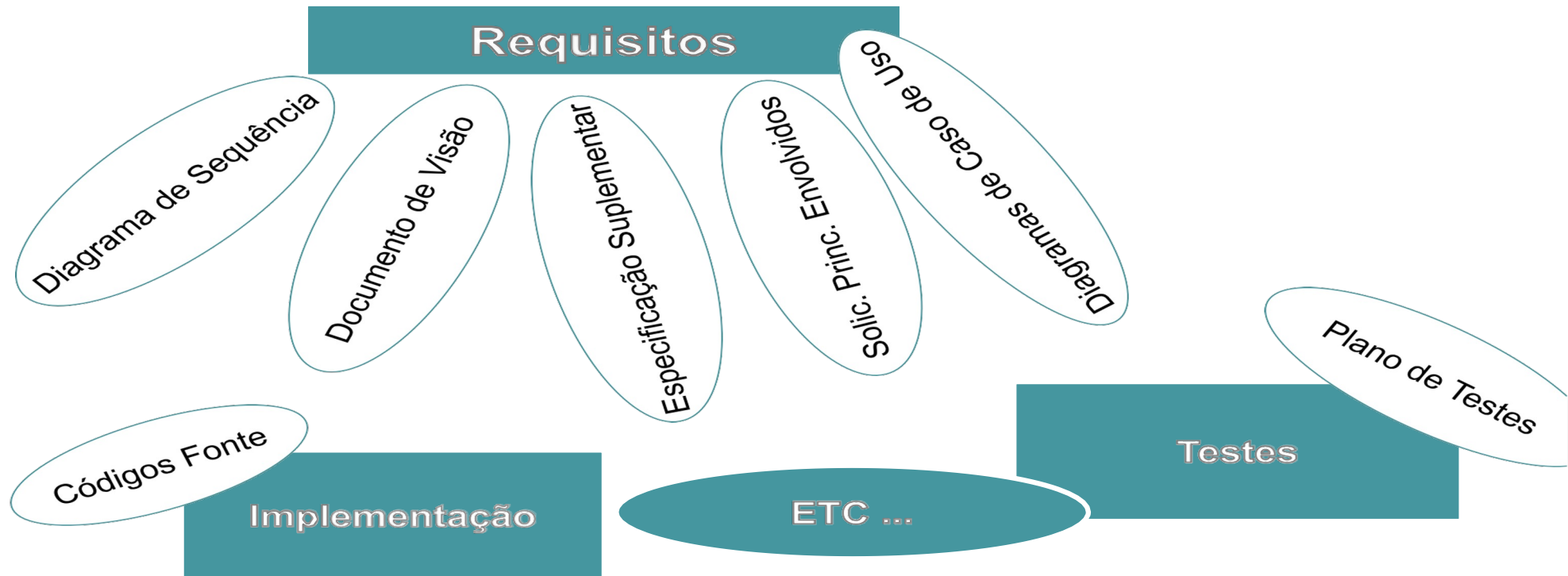
# Controles e Estratégias de Versionamento com GIT

Uma característica das mais importantes do software é: Sempre sofre modificações.

Quando o software deixa de ser modificado?



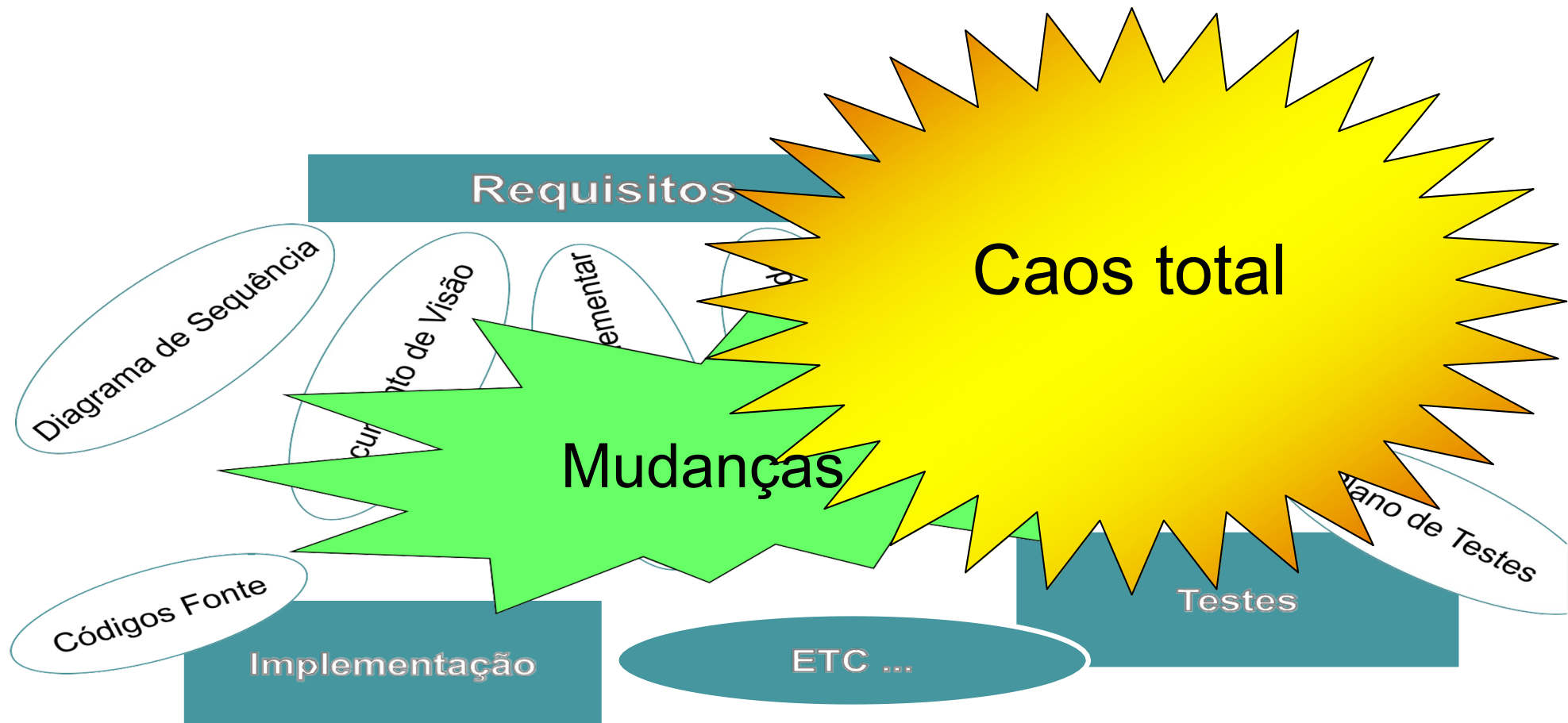
# Controles e Estratégias de Versionamento com GIT



# Controles e Estratégias de Versionamento com GIT



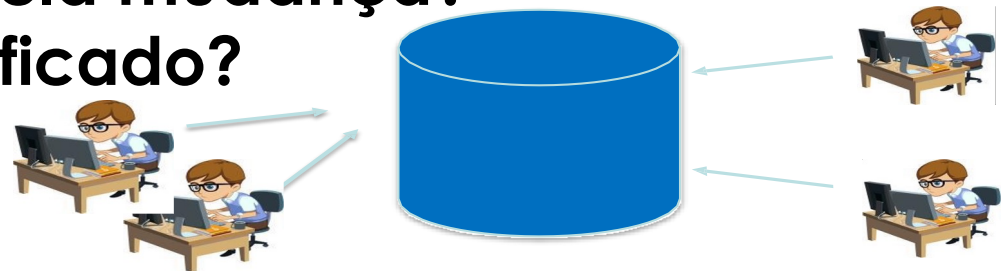
# Controles e Estratégias de Versionamento com GIT



# Motivação

- Falta de sincronismo entre as atividades.
- Notificação limitada.
- Conflito entre as atividades paralelas.
- Falta de controle de modificações.

Em que versão foi realizada a correção?  
Qual é a versão mais atual?  
Quem foi o responsável pela mudança?  
O que realmente foi modificado?  
Quando?



# Sistemas de Controle de Versão

**Os sistemas de controle de versão são ferramentas de software que ajudam as equipes de software a gerenciar as alterações ao código-fonte ao longo do tempo.**

**Como os ambientes de desenvolvimento aceleraram, os sistemas de controle de versão ajudam as equipes de software a trabalhar de forma mais rápida e inteligente.**



# Sistemas de Controle de Versão

Para quase todos os projetos de software, o código-fonte é como uma **mina de ouro**: um bem precioso com valor deve ser protegido.

Para a maioria das equipes de software, o código-fonte é um repositório de **conhecimento inestimável** do domínio do problema que os desenvolvedores coletaram e aprimoraram com muito cuidado.



# Sistemas de Controle de Versão

- Controlam o evolução do código do projeto.
- Possibilitam encontrar e reverter problemas ou alterações incluídos no código facilmente.
- O controle de versão protege o código-fonte contra catástrofes e a possível degradação causada por erro humano e consequências ruins.
- Sua eficácia é comprovada pela exigência em certificações como CMMI.

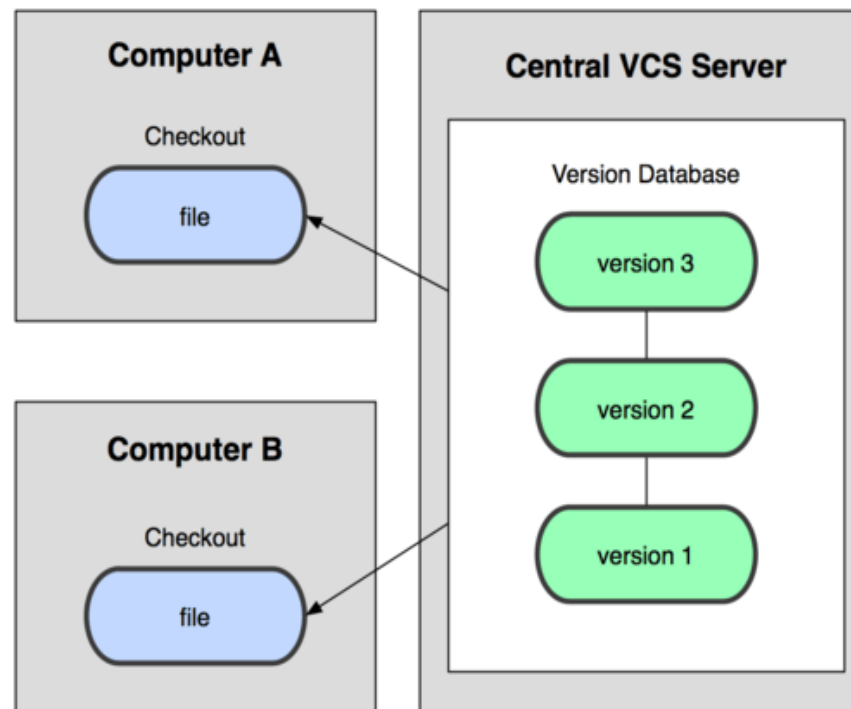
# Benefícios

- Controle do histórico.
- Trabalho em equipe.
- Marcação e resgate de versões estáveis.
- Ramificação do projeto.



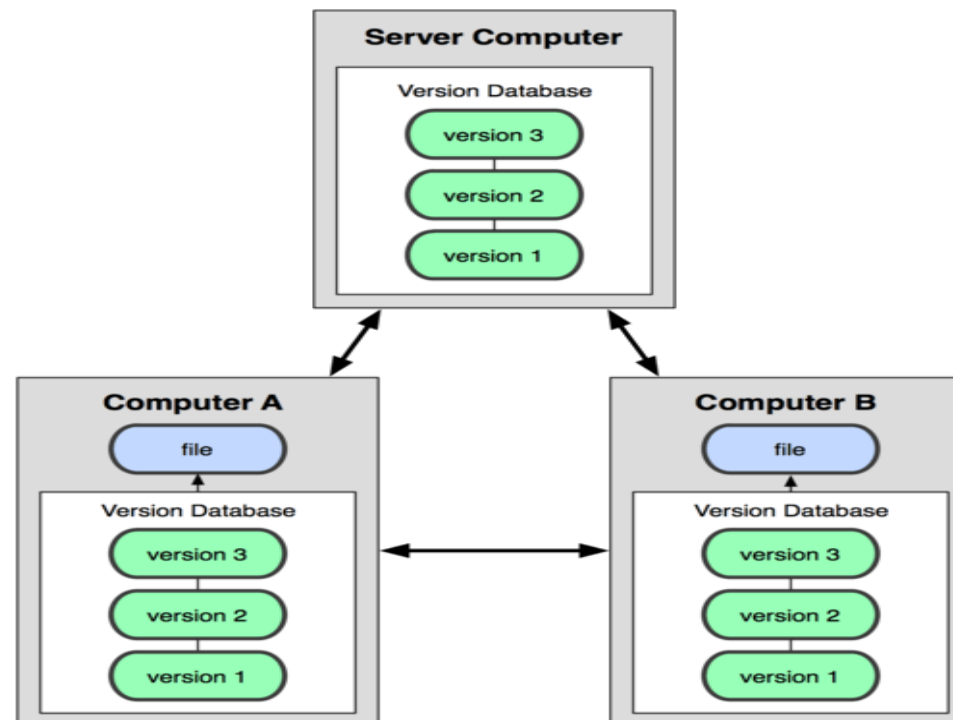
# Controle de Versão com Git / Métodos de Controle de Versão

## Centralizado



# Controle de Versão com Git / Métodos de Controle de Versão

## Distribuído



# Sistemas de Controle de Versão

- CVS(1986) → SVN(2004) → Git(2005);
- Team Foundation Server(2005) / ClearCase(1992) (Comercial).

# O Git

- Linus Torvalds;
- 2005;
- Controlar o código fonte do kernel Linux.



# O Git - Critérios para o Desenvolvimento

- Não ser igual ao CVS.
- Suportar fluxo distribuído.
- Proteção contra corrompimento (acidente ou maldoso).
- Alta performance.

# Pré-Requisitos

- Ter o Git instalado na máquina;
- Um repositório exclusivo para cada projeto.





# Inicializando o repositório



```
~/repositorio: git init
```

# Verificando o repositório



~/repositorio: git status

# Rastreando o Arquivo



```
~/repositorio: git add [nome_arquivo]
```

# Salvando o trabalho localmente



```
~/repositorio: git commit -m "[descricao]"
```

# Anatomia de um Commit de Qualidade

Um commit não é apenas um backup de segurança. É uma mensagem para o futuro.

Um commit deve ser atômico (fazer apenas uma coisa) e conter uma mensagem imperativa e descritiva.

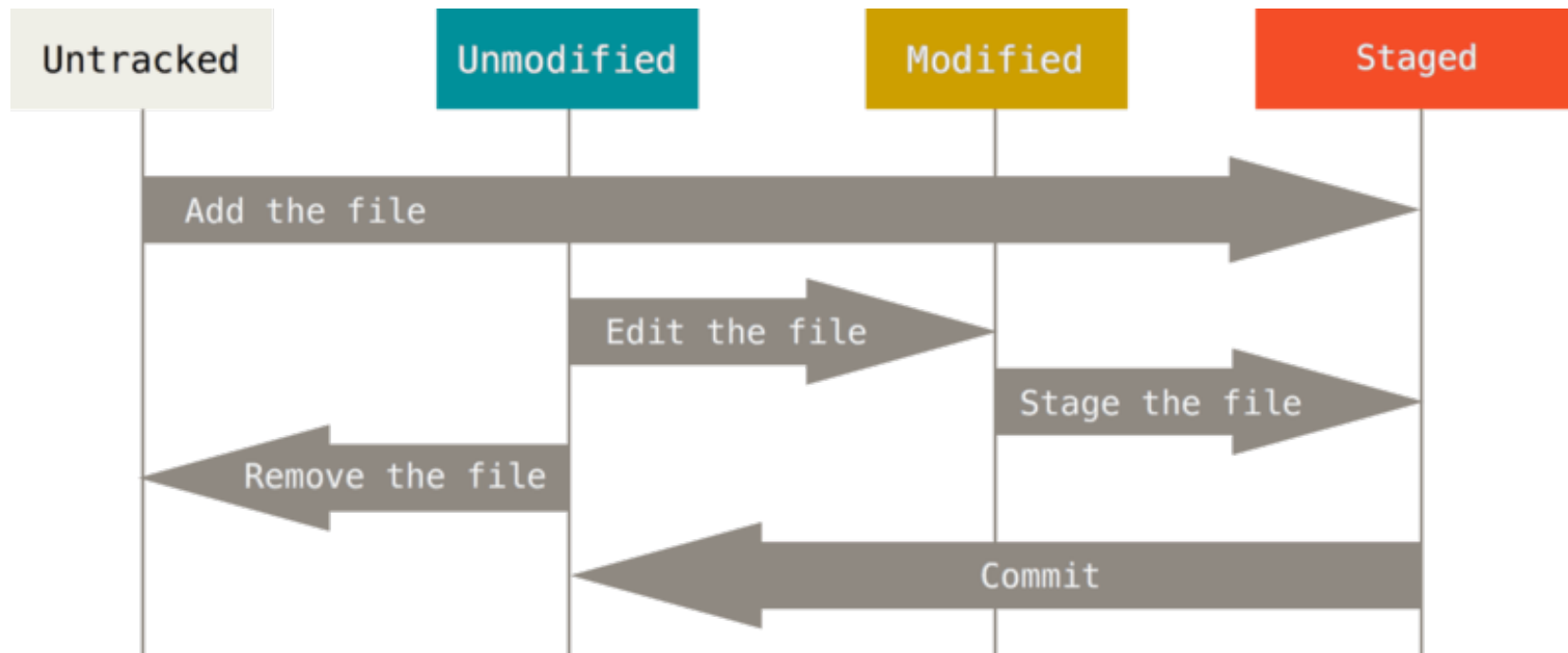
Em ambientes corporativos, é fundamental integrar a rastreabilidade.

Exemplo de excelência: `git commit -m "FIN-432: Adiciona validação de saldo no serviço de autorização"`.

Mensagens como "Fix" ou "Agora vai" destroem o pilar de Partilha (Sharing) do DevOps, pois impedem que a equipa compreenda a evolução do sistema.



# Estados dos Arquivos



# Salvando o trabalho localmente



~/repositorio: git log

# Salvando o trabalho localmente



~/repositorio: git diff

~/repositorio: git diff -- [file\_name]



# COLABORAÇÃO

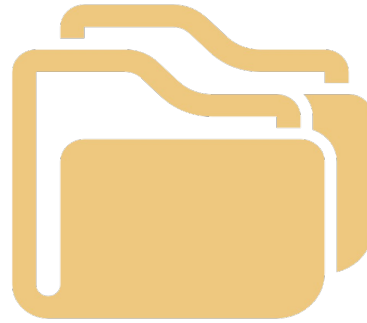


Prof. Anderson Augusto Bosing

# Pré-Requisitos

- Um meio de transporte para o remoto (system file, HTTP, SSH);
- Uma chave de acesso ao remoto (quando necessário).

# Clonando um repositório remoto



```
~/repositório_local: git clone [repo_system_url]
```

# Enviando mudanças para o repositório



```
~/repositório_local: git push
```

# Trazendo mudanças do repositório



```
~/repositório_local: git push
```

# Merge – Verificando conflito



Unmerged paths:  
(use "git add <file>..." to mark resolution)

both modified:      bemvindo.txt

# Merge – Interpretando conflito



Bem vindo ao mundo Git!

<<<<<< HEAD

alteracao?

=====

alteracao!

>>>>>> f8b208abf464823bd5f1eb6a4fcaabb9a26f8f7b

# Merge – Após resolver conflito



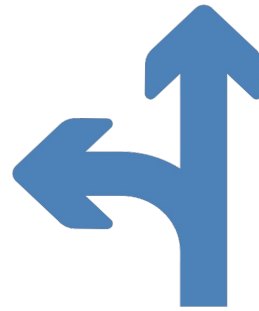
```
~/repositório_local: git add [nome_arquivo]
```



# Ramificações

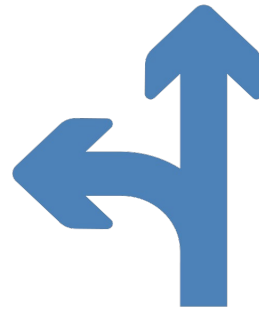
- Ramificações auxiliam no isolamento de diferentes trabalhos simultâneos;
- Ramos de desenvolvimento devem ser criados para garantir que o código fonte do trabalho atual tem como base uma versão estável e funcionando, podendo ser integrada de volta sem maiores problemas;
- Adiciona complexidade e necessidade de atenção por parte dos desenvolvedores;
- Garante que experiências possam ser feitas facilmente sem perder o trabalho que está incompleto em um ticket.

# Manipulando ramificações



~/repositorio: `git checkout -b [identificador_branch]` / criar  
~/repositorio: `git branch` / listar  
~/repositorio: `git branch -d [identificador_branch]` / deletar / !!!  
~/repositorio: `git checkout [identificador_branch]` / navegar

# Manipulando ramificações



```
~/repositorio: git merge [identificador_branch]
```

# Acabaram os comandos ?

Não.

Link para consulta de comandos.

[https://training.github.com/downloads/pt\\_BR/github-git-cheat-sheet/](https://training.github.com/downloads/pt_BR/github-git-cheat-sheet/)



# O Diretório Oculto: .git

O arquivo `.gitignore` é um arquivo de texto que diz ao Git quais arquivos ou pastas ele deve ignorar em um projeto.

Para criar um arquivo `.gitignore`, crie um arquivo de texto e dê a ele o nome de `.gitignore` (lembre-se de incluir o `.` no começo). Em seguida, edite esse arquivo conforme necessário. Cada nova linha deve listar um arquivo ou pasta adicional que você quer que o Git ignore.

# Arquivo .gitignore

O arquivo .gitignore é um arquivo de texto que diz ao Git quais arquivos ou pastas ele deve ignorar em um projeto.

Para criar um arquivo .gitignore, crie um arquivo de texto e dê a ele o nome de .gitignore (lembre-se de incluir o . no começo). Em seguida, edite esse arquivo conforme necessário. Cada nova linha deve listar um arquivo ou pasta adicional que você quer que o Git ignore.

# Arquivo .gitignore

As entradas neste arquivo também podem seguir um padrão de correspondência.

**\*** é usado para corresponder a um caractere curinga .

**/** é usado para ignorar nomes de caminhos relativos ao arquivo .gitignore .

**#** é usado para adicionar comentários ao arquivo .gitignore.



# Exemplo.gitignore para Java

**# Compiled class file**  
**\*.class**

**# Log file**  
**\*.log**

**# BlueJ files**  
**\*.ctxt**

**# Mobile Tools for Java (J2ME)**  
**.mtj.tmp/**

**# Package Files #**  
**\*.jar**  
**\*.war**  
**\*.nar**  
**\*.ear**  
**\*.zip**  
**\*.tar.gz**  
**\*.rar**

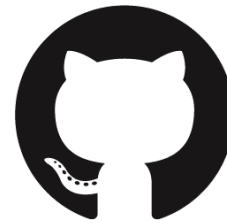




# Git e Github são a mesma coisa ?



**git**



**GitHub**

# DEMONSTRAÇÃO PRÁTICA

- Criar conta no github.
- Criar repositório.
- Clonar repositório.
- Criar branches.
- Adicionar arquivos.
- Merge

# VAMOS PRATICAR ?

Em caso de incêndio



- 1. git commit
- 2. git push
- 3. saia do prédio



# O Modelo Clássico: GitFlow

O GitFlow (criado por Vincent Driessen) é estruturado e robusto.

Utiliza duas ramificações de longa duração: master/main (o histórico oficial de produção) e develop (a linha de integração de funcionalidades).

Para apoiar o ciclo de vida, ramifica-se em:

feature/\*: para novos desenvolvimentos (sai de develop e volta para develop).

release/\*: para estabilizar versões antes do lançamento.

hotfix/\*: criadas a partir de master para corrigir problemas urgentes, regressando tanto para a master como para develop.

É ideal para aplicações monolíticas com ciclos de lançamento fixos (ex: a cada mês).



# O Fluxo Ágil: GitHub Flow

O GitHub Flow remove a ramificação develop e aposta numa simplicidade orientada para implantações frequentes (deployments).

O princípio é: a ramificação principal (main) é estritamente imutável para commits diretos e deve estar sempre implantável (deployable).

Toda a nova alteração, seja bugfix ou funcionalidade, origina-se da main para uma ramificação curta, e entra após revisão e validação.

Como a main é estável por definição, toda a validação de esteira CI é feita antes da fusão.



# DevOps

**Aula: Laboratório Prático -  
CI e Qualidade de Código  
(Mão na Massa)**



Prof. Anderson Augusto Bosing

# A Missão do Squad e Divisão de Papéis

A engenharia de software não se faz sozinho.

O laboratório será executado em grupos de exatos 2 ou mais membros, ideal são 5.

O objetivo é criar o repositório base para o MVP de um SaaS de gestão de vendas recorrentes.

**Membro 1 (Tech Lead):** Criará o repositório no GitHub, adicionará os colegas e configurará a proteção da branch.

**Membros 2 e 3 (Desenvolvedores):** Criarão a lógica de negócio (API de Pedidos) e os testes automatizados.

**Membros 4 e 5 (DevOps/QA):** Configurarão a pipeline do GitHub Actions e realizarão o Code Review (Revisão de Código) para aprovar o Merge.

**Tópico de Apoio / Discussão:** Esta divisão simula a dinâmica de responsabilidade compartilhada e a cultura de revisão por pares.



# Passo 1 - O Repositório e a Proteção do Tronco (Main/Master)

**Conteúdo (Ação do Membro 1):**

- 1. Crie um repositório público no GitHub chamado pedido-api.**
- 2. Vá em Settings > Collaborators e convide os outros 4 membros.**
- 3. Vá em Settings > Branches > Add branch protection rule.**
- 4. Defina o padrão como main.**
- 5. Regras Obrigatórias: Marque "Require a pull request before merging" (exigir 1 aprovação) e "Require status checks to pass before merging".**

**Tópico de Apoio / Discussão:** A partir de agora, ninguém do grupo (nem o Tech Lead) consegue fazer um git push direto para a main. O código é forçado a passar pela esteira.





# Passo 2 - O Esqueleto da Aplicação (Spring Initializr)

Acesse [start.spring.io](https://start.spring.io).

Configure um projeto Maven, Java 17 (ou 21) e empacotamento Jar.

Adicione as dependências: Spring Web, Spring Data JPA, Validation e o PostgreSQL Driver (preparando a arquitetura para o deploy futuro em plataformas como o Render).

Gere o projeto, extraia na máquina local, faça o `git init`, `git add .`, `git commit` e force o primeiro push para inicializar o repositório (antes das regras de proteção ativarem totalmente, ou via PR de fundação).



# Passo 3 - A Lógica de Negócio e Testes Unitários

Conteúdo (Ação dos Membros 2 e 3): Clonem o repositório e criem uma ramificação de trabalho: `git checkout -b feature/validacao-fatura`.

Criem uma classe `PedidoService.java` com um método boolean `validarPedido(Double valor)`. A regra de negócio exige que o valor seja estritamente maior que zero. Criem a classe de teste `PedidoService Test.java` usando JUnit.

Escrevam dois testes: um passando o valor 100.0 (deve retornar true) e outro passando -5.0 (deve retornar false ou lançar exceção).

Tópico de Apoio / Discussão: Sem o teste unitário, a pipeline de CI é apenas um compilador glorificado. O teste é o "Andon" que trava a esteira se houver falha.



# Passo 3 - A Lógica de Negócio e Testes Unitários

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class FaturaService {
```

```
    public boolean validarFatura(Double valor) {
```

```
        if (valor == null || valor <= 0) {
```

```
            throw new IllegalArgumentException("O valor da fatura deve ser maior que zero.");
```

```
        }
```

```
        return true;
```

```
    }
```

```
}
```



# Passo 3 - A Lógica de Negócio e Testes Unitários

```
class FaturaServiceTest {  
  
    private final FaturaService service = new FaturaService();  
  
    @Test  
    void deveValidarFaturaComValorPositivo() {  
        assertTrue(service.validarFatura(100.0));  
    }  
  
    @Test  
    void deveLancarExcecaoParaFaturaInvalida() {  
        Exception exception = assertThrows(IllegalArgumentException.class, () -> {  
            service.validarFatura(-5.0);  
        });  
        assertEquals("O valor da fatura deve ser maior que zero.", exception.getMessage());  
    }  
}
```



# Passo 4 - Configurando a Pipeline (GitHub Actions)

**Conteúdo (Ação dos Membros 4 e 5):** Na mesma branch feature/validacao-pedido, criem a estrutura de pastas: `.github/workflows/`.

**Criem o arquivo ci.yml com o manifesto YAML.**

**Ele deve reagir a Push e Pull Requests na main. O trabalho (Job) deve conter os seguintes passos:**

**actions/checkout@v3** (Para baixar o código).

**actions/setup-java@v3** (Configurar a JDK escolhida).

**Run Maven Build:** Executar o comando `mvn clean verify`.

**Tópico de Apoio / Discussão:** O comando `verify` garante que o Maven compila e corre a bateria completa de testes automatizados no servidor efêmero do GitHub.



# Passo 4 - Configurando a Pipeline (GitHub Actions)

name: CI - Pedido API

on:

push:

branches: [ "main" ]

pull\_request:

branches: [ "main" ]

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout do Código

uses: actions/checkout@v3

- name: Configurar JDK 17

uses: actions/setup-java@v3

with:

java-version: '17'

distribution: 'temurin'

cache: maven

- name: Build e Testes com Maven

run: mvn clean verify



# Passo 5 - A Dinâmica de Integração (Pull Request)

O Squad junta as peças:

Executam `git add .`, `git commit -m "feat: validação de fatura e CI"` e fazem o push.

No GitHub, abrem um Pull Request (PR) da ramificação atual contra a main.

A Magia do DevOps: O GitHub Actions interceta o PR automaticamente. O ecrã mostrará o Job a executar o Java e o Maven.

Tópico de Apoio / Discussão: Se um dos programadores tiver colocado um bug na regra do valor da fatura, a bolinha ficará vermelha e o botão de Merge ficará bloqueado pela regra que o Tech Lead configurou.



# Passo 6 - Code Review e Encerramento

O trabalho de auditoria final:

Com a pipeline verde (testes aprovados), o membro DevOps/QA acede à aba "Files Changed" no PR.

Avalia a qualidade da classe Java e clica em "Approve". O bloqueio é levantado.

A equipa clica em "Merge Pull Request". O código está formalmente integrado no tronco (main).

Tópico de Apoio / Discussão: O erro humano foi mitigado pela máquina (Testes/CI) e validado pela revisão de pares. O Squad construiu a primeira etapa segura de um fluxo de valor transacional contínuo.





# DevOps

**Aula: Entrega Contínua,  
Estratégias de Deploy e  
Containerização**



Prof. Anderson Augusto Bosing

# A Fronteira entre CI e CD

A Integração Contínua (CI) garante que o código compila e passa nos testes automatizados, terminando na geração de um artefacto (ex: um .jar de uma aplicação Spring Boot). A Entrega Contínua (CD) assume a partir daí. É a automação do processo que pega nesse artefacto validado, configura o ambiente, injeta as variáveis sensíveis e coloca o sistema no ar de forma consistente.

O CI constrói a bomba; o CD é o sistema de mísseis guiados que a entrega no alvo certo.

# Continuous Delivery vs. Continuous Deployment

O acrónimo "CD" possui dois níveis de maturidade técnica:

**Continuous Delivery (Entrega Contínua):** O código está sempre pronto para produção. O pipeline automatiza tudo até à homologação, mas o deploy produtivo exige a aprovação de um humano (um clique).

**Continuous Deployment (Implantação Contínua):** Toda a alteração que passa nos testes é implantada automaticamente em produção sem intervenção humana.



# O Paradoxo do Deploy Manual e o "Downtime"

O modelo legado, implantar implicava indisponibilidade (Downtime).

O administrador parava o serviço, substituíam os binários e reiniciava. Numa arquitetura de vendas recorrentes ou fintech, 10 minutos de inatividade durante um pico transacional causam perda de dezenas de autorizações e acumulam falhas na porta de entrada da API.

Tópico de Apoio / Discussão: A implantação automatizada procura ser invisível para o cliente final.



# Desacoplar "Deploy" de "Release"

Um dos maiores saltos culturais do CD é separar a instalação física do código (Deploy) da libertação da funcionalidade para o utilizador (Release).

Deploy é um evento técnico;

Release é uma decisão de negócio.

Tópico de Apoio / Discussão: A engenharia pode fazer 50 deploys por dia, mas o Marketing pode decidir que a funcionalidade só será ativada ("Release") na sexta-feira.

# Feature Flags (Interruptores de Código)

As Feature Flags (ou Toggles) são condicionais lógicas injetadas na aplicação (ex: `if(pix_feature_enabled)`).

Permitem que a equipa faça o merge de código diretamente para a linha principal (main), mantendo a funcionalidade latente/oculta em produção.

Tópico de Apoio / Discussão: Separa o risco de engenharia do risco de lançamento do produto.



# O Ciclo de Vida e o Débito Técnico das Flags

As Feature Flags são poderosas, mas perigosas se esquecidas. Uma flag permanente torna-se um débito técnico.

O código passa a ter múltiplos caminhos de execução difíceis de testar.

A boa prática DevOps exige que, após a validação da funcionalidade em produção, um novo ticket seja aberto no backlog exclusivamente para remover a condicional e apagar o código antigo.

Tópico de Apoio / Discussão: Manter código "zumbi" em produção é uma vulnerabilidade de arquitetura.



# DevOps

## Aula: Estratégias de Deploy Avançadas



Prof. Anderson Augusto Bosing



# Estratégias de Deploy: Rolling Update

Numa arquitetura com múltiplos nós, o Rolling Update atualiza a aplicação gradualmente.

O pipeline retira um servidor do balanceador de carga, atualiza-o, verifica a sua saúde (Healthcheck), recoloca-o a receber tráfego e avança para o próximo. O serviço nunca fica indisponível (Zero Downtime).

Tópico de Apoio / Discussão: O desafio é que, durante a transição, a Versão 1 e a Versão 2 da aplicação operam em simultâneo.



# Estratégias de Deploy: Blue/Green Deployment

No Blue/Green Deployment, mantêm-se dois ambientes de produção completos. O "Blue" é o ativo. O "Green" é um clone inativo.

Implanta-se a versão nova no Green de forma isolada. Quando validado, o roteador redireciona 100% do tráfego instantaneamente do Blue para o Green.

Tópico de Apoio / Discussão: Se ocorrer uma falha, o rollback é imediato (questão de segundos). Contudo, custa o dobro em infraestrutura de computação.



# Estratégias de Deploy: Canary Release

Inspirado nos "canários nas minas de carvão", o Canary Release mitiga o risco testando com utilizadores reais.

A nova versão recebe apenas uma pequena percentagem do tráfego (ex: 5%). Ferramentas de observabilidade (APMs) monitorizam a taxa de erro.

Se o canário sobreviver (sem latência), o pipeline aumenta gradualmente a carga (20%, 50%, 100%).

Tópico de Apoio / Discussão: A abordagem mais segura para ecossistemas de alto volume.



# Dark Traffic (Tráfego Sombra)

Para validações extremas (ex: reescrita de um módulo inteiro ou processo de risco), usa-se o Shadow Traffic.

O balanceador de carga duplica as requisições recebidas em produção.

A requisição real é processada pelo sistema legado e devolvida ao cliente.

A requisição "sombra" é processada pelo novo sistema em modo silencioso, apenas para comparar resultados e testar performance, sem impactar a resposta real.

Tópico de Apoio / Discussão: Permite testar cargas de Black Friday antes de substituir a tecnologia.



# O Ponto de Falha: O Banco de Dados

Pode ter a melhor estratégia de Blue/Green, mas se a sua aplicação depende de um PostgreSQL, o banco de dados geralmente é um só.

Se o deploy da nova versão apaga uma coluna essencial para a versão antiga, o sistema "Blue" que ainda está ativo irá colapsar (Erro 500).

Tópico de Apoio / Discussão: Alterações em bases de dados relacionais são o maior obstáculo para deploys de Zero Downtime.



# O Padrão Expand & Contract (Bancos Relacionais)

Para resolver o conflito de base de dados, divide-se a migração em etapas retrocompatíveis. Em vez de renomear uma coluna nome para nome\_completo num único passo destrutivo:

**Expandir:** Cria-se a nova coluna vazia. O código V1 continua a ler a antiga.

**Migrar:** O código V2 escreve nas duas colunas simultaneamente. Executa-se um script (usando ferramentas como Flyway ou Liquibase) para copiar os dados históricos para a nova coluna.

**Contrair:** Numa release futura, quando a V1 já não existir, apaga-se a coluna antiga.